ChadWallet — Assignment Summary

Links

Live Demo: https://chad-wallet-omega.vercel.app

GitHub Repository: https://github.com/emam07/ChadWallet

Design Approach

ChadWallet is the landing page for a non-custodial Solana trading wallet, built with Next.js 15 (App Router), React 19, TypeScript, and Tailwind. The goal was a site that feels like a real product rather than a static brochure, so the marketing sections are backed by live market data and a working trade view.

The page is composed of focused, self-contained components (Hero, Features, Market Preview, Stats, How It Works, Trade panel, CTA), each driven by the same dark glassmorphism design system and Framer Motion animations. All market data flows through a small set of API routes that talk to free, key-less public providers (DexScreener and GeckoTerminal), with a deterministic mock fallback so the app never breaks when a provider rate-limits. The whole project runs with zero paid API keys.

AI Tools Used

- **Claude Code** — primary tool. Used for the API layer, the security/test suite, debugging deploy issues, and most of the iterative refactoring.
- **ChatGPT** — quick research and sanity checks: Solana token decimals, Privy v3 config, GraphQL/REST field shapes.
- **Cursor** — inline edits and fast component tweaks while building out the UI.

Development Timeline

- **Research** (~half day) — Solana data providers, charting libraries, auth options.
- **UI** (~2 days) — full landing page, design system, animations, responsive layout.
- **Trading Page** (~1.5 days) — live charts, trades feed, swap panel with Jupiter quotes.
- **Authentication** (~1 day) — Privy login/signup, provider config, CSP wiring.
- **Testing & hardening** (~1 day) — validation, rate limiting, security headers, ~650 tests.

Issues Encountered

Login worked locally but failed on the deployed site. Privy was being asked to create a client-side embedded wallet that the app's dashboard config doesn't allow; the mismatch only surfaced over HTTPS. Fixed by matching the code to the dashboard (`createOnLogin: "off"`) and ensuring the `NEXT_PUBLIC` app ID was set in the deploy environment.

Trending list was too short. GeckoTerminal's pools endpoint caps at 100 pools and many share base tokens, yielding only ~40 unique tokens. Fixed by merging two live sources in parallel and de-duplicating by address and symbol, ranked by 24h volume — now a stable top-~50 that never collapses to the mock.

Swap "You receive" amount was wrong. The original math assumed every token had 9 decimals and ignored SOL's USD price. Reworked it to estimate both legs in USD so the result is decimal-agnostic, while still using Jupiter for the live price-impact figure.

Stale build chunks after deploy. Occasional `ChunkLoadError` traced to a stale `.next` cache rather than a code bug; resolved with a clean rebuild and a couple of build-hygiene fixes (favicon, metadata).

Future Improvements

- Wire real swap execution and on-chain positions (web3.js + Privy signing + RPC reads).
- Centralize duplicated helpers like `formatPrice` into a shared module.
- Add a small caching layer in front of the public APIs to cut latency and rate-limit risk.
- Expand E2E coverage of the OAuth login flow, which currently can't be fully automated.

Notes

The codebase favors honesty over decoration: where live data isn't available on the key-less tier (e.g. holder counts), the UI shows a clear empty state instead of faked numbers. Validation, rate limiting, and security headers are in place across every API route, covered by ~650 automated tests.